WUOLAH

Examen-reviews.pdf

candela_pr88

Introducción a la Ingeniería del Software y los Sistemas de Información II

2º Grado en Ingeniería Informática - Ingeniería del Software

Escuela Técnica Superior de Ingeniería Informática
Universidad de Sevilla

Accede al documento original

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio

Necesito concentración

ali ali oooh esto con 1 coin me lo quito yo...

WUOLAH

Examen DeliverUS - Modelo A - Marzo (Reviews)

Se ha incorporado un nuevo requisito en la aplicación DeliverUS, que permite a los clientes dejar reviews sobre los restaurantes en los que han realizado pedidos. Además se podrá consultar la valoración media que los clientes tienen sobre los restaurantes. En DeliverUS, las reviews representan la valoración de los clientes sobre un restaurante.

Requisitos funcionales a implementar

RF1. Listado de reviews de un restaurante: Como usuario, quiero listar las reviews de un restaurante, para ver las valoraciones de otros clientes.
`GET /restaurants/:restaurantId/reviews`

RF2. Creación de una review: Como cliente, quiero dejar una review en un restaurante en el que haya pedido, para valorar mi experiencia.
`POST /restaurants/:restaurantId/reviews`

RF3. Edición de una review: Como cliente, quiero poder editar mi review, para corregir o actualizar mi opinión.
`PUT /restaurants/:restaurantId/reviews/:reviewId`

RF4. Eliminación de una review: Como cliente, quiero poder eliminar mi review, para retirar mi opinión si ya no es relevante.
`DELETE /restaurants/:restaurantId/reviews/:reviewId`

RF5. Cálculo de la media de estrellas de un restaurante: Como usuario, quiero ver la media de puntuaciones de un restaurante, para conocer su valoración general.
El atributo virtual avgStars debe calcularse como el promedio de todas las reviews. El objeto json que representa al restaurante tendrá la forma siguiente (nótese que se incluye la propiedad avgStars). Si el restaurante no tiene reviews, avgStars debe ser null o undefined.

Antes de nada ejecutar en la terminal: `npm run install:backend`

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

1. Migraciones, modelos y cambios necesarios (2 puntos)

Cree y modifique las migraciones necesarias para implementar el modelado conceptual así como cree y modifique los modelos necesarios. Complete los ficheros /src/database/migrations/20250723214455-create-review.js y /src/models/Review.js, además de las modificaciones necesarias sobre otros ficheros.

Migraciones de review:

```
id: {
   allowNull: false,
   autoIncrement: true,
   primaryKey: true,
   type: Sequelize.INTEGER
 },
 stars: {
   allowNull:false,
   type: DataTypes.INT
 },
 body: {
   allowNull:true,
   type: DataTypes.TEXT
 },
 restaurantId: {
   allowNull: false,
   type: Sequelize.INTEGER,
   onDelete: 'CASCADE', // Se borran las reseñas de los restaurantes eliminados
   references: {
     model: {
       tableName: 'Restaurants'
     },
     key: 'id'
   }
 },
 customerId: {
   allowNull: false,
   type: Sequelize.INTEGER,
   onDelete: 'CASCADE', // Se borran las reseñas de los usuarios eliminados
   references: {
     model: {
       tableName: 'Users'
     },
     key: 'id'
   }
 }
```

Modelo de review:

Definimos las claves ajenas:

```
Review.belongsTo(models.Restaurant, { foreignKey: 'restaurantId', as: 'restaurant' })
Review.belongsTo(models.User, { foreignKey: 'customerId', as: 'user' })
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

Definimos la tabla review:

```
Review.init({
   id: {
     allowNull: false,
     autoIncrement: true,
     primaryKey: true,
     type: Sequelize.INTEGER
   },
   stars: {
     allowNull:false,
     type: DataTypes.INT
   },
   body: {
     allowNull:true,
     type: DataTypes.TEXT
   },
   restaurantId: {
     allowNull: false,
     type: Sequelize.INTEGER,
     onDelete: 'CASCADE',
     references: {
       model: {
         tableName: 'Restaurants'
       },
       key: 'id'
     }
   },
   customerId: {
     allowNull: false,
     type: Sequelize.INTEGER,
     onDelete: 'CASCADE',
     references: {
       model: {
         tableName: 'Users'
       },
       key: 'id'
     }
   }
 }
```

Añadimos la clave ajena en el modelo de restaurante (/src/models/Restaurant.js):

```
Restaurant.hasMany(models.Review, { foreignKey: 'restaurantId', as: 'reviews' })
```

Añadimos la clave ajena en el modelo de usuario (/src/models/User.js):

```
User.hasMany(models.Review, { foreignKey: 'customerId', as: 'reviews' })
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio

Necesito concentración

ali ali oooh esto con 1 coin me lo quito yo...

WUOLAH

2. Rutas de Review (1 punto)

Implemente las siguientes rutas:

```
RF1: GET /restaurants/:restaurantId/reviews
RF2: POST /restaurants/:restaurantId/reviews
RF3: PUT /restaurants/:restaurantId/reviews/:reviewId
RF4: DELETE /restaurants/:restaurantId/reviews/:reviewId
```

Se le proporciona el fichero /src/routes/ReviewRoutes.js para definir estas rutas y recuerde incorporar los middlewares necesarios en cada ruta.

Rutas de review (ir mirando los requisitos funcionales necesarios para cada ruta, en ReviewMiddleware.js se encuentran definidas las funciones para verificar los requisitos):

```
const loadReviewRoutes = function (app) {
    app.route('/restaurants/:restaurantId/reviews')
    .get(
      checkEntityExists(Restaurant, 'restaurantId')),
      ReviewController.index
    .post(
      isLoggedIn,
      hasRole('customer'),
       checkEntityExists(Restaurant, 'restaurantId'),
       userHasPlacedOrderInRestaurant,
      checkCustomerHasNotReviewed,
      ReviewValidation.create,
      handleValidation,
      ReviewController.create
    )

    app.route('/restaurants/:restaurantId/reviews/:reviewId ')
    .put(
      isLoggedIn,
      hasRole('customer'),
      checkReviewOwnership,
      checkEntityExists(Restaurant, 'restaurantId'),
      checkEntityExists(Review, 'reviewId'),
      checkReviewBelongsToRestaurant,
      ReviewValidation.create,
      handleValidation,
      ReviewController.update
    )
    .delete(
      isLoggedIn,
      hasRole('customer'),
      checkReviewOwnership,
      checkEntityExists(Restaurant, 'restaurantId'),
      checkEntityExists(Review, 'reviewId'),
      checkReviewBelongsToRestaurant,
      ReviewController.destroy
    )
}
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

3. Validaciones para Review (1 puntos)

Implemente las reglas de validación para la creación y la edición de una review. Se le proporciona el fichero /src/controllers/Validation/ReviewValidation.js para definir estas validaciones.

```
const create = [
   check('stars').exists().isInt({ min: 0, max: 5 }),
    check('body').optional({nullable:  true,  checkFalsy:  true}).isString().isLength({
min: 1, max: 255 }).trim()
]
const update = [
     check('stars').exists().isInt({ min: 0, max: 5 }),
     check('body').optional({nullable:  true,  checkFalsy:  true}).isString().isLength({
min: 1, max: 255 }).trim()
]
```

4. Controlador de Review (2 puntos)

Implemente las funciones necesarias para RF1, RF2, RF3 y RF4. Se le proporciona el fichero /src/controllers/ReviewController.js para definir estas funciones.

```
const ReviewController = {
  // Controller para listar las reviews (GET)
    async index (req, res) {
    try{
      const reviews = await Review.findAll({
        where: { restaurantId: req.param.restaurantId },
        order: [['createdAt', 'DESC']]
        }
      )
      res.json(reviews)
    }
      catch(err) {
      res.status(500).send(err)
    }
  },

  // Controller para publicar una nueva review (POST)
  async create (req, res) {
    try{
      const newReview = Review.build(req.body)
      newReview.customerId = req.user.id
      newReview.restaurantId = req.param.restaurantId
      const createdReview = await newReview.save()
      res.json(createdReview)
    }
      catch(err) {
      res.status(500).send(err)
    }
  },
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	PLAN TURBO	PLAN PRO	PLAN PRO+
Descargas sin publi al mes	10	40	80
Elimina el video entre descargas	✓	✓	✓
Descarga carpetas	✗	✓	✓
Descarga archivos grandes	✗	✓	✓
Visualiza apuntes online sin publi	✗	✓	✓
Elimina toda la publi web	✗	✗	✓
Precios Anual	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo, ¿Qué nota vas a sacar?

WUOLAH

Oferta limitada en el tiempo. Los planes están sujetos a los términos y condiciones que puedes consultar en **wuolah.com/terms**

```
  // Controller para actualizar una review (UPDATE)
  async update (req, res) {
    try{
      const currentReview = await Review.findByPk(req.param.reviewId)
      currentReview.stars = req.body.stars
      currentReview.body = req.body.bodyconst
      res.json(currentReview)
    }
    catch(err) {
      res.status(500).send(err)
    }
  },

  // Controller para eliminar una review (DELETE), siempre igual en todos!!
  async destroy (req, res) {
    try {
    const result = await Review.destroy({ where: { id: req.params.reviewId } })
    let message = ''
    if (result === 1) {
      message = 'Sucessfuly deleted review id.' + req.params.reviewtId
    } else {
      message = 'Could not delete review.'
    }
    res.json(message)
  } catch (err) {
    res.status(500).send(err)
  }
  }

}
```

5. Comprobación de que el cliente puede crear una review (2 puntos)

Implemente las dos comprobaciones que un cliente debe cumplir para publicar una review, esto es:

- **Tiene al menos un pedido en el restaurante en el que quiere poner una review.**
- **No tiene ya una review publicada para ese restaurante.**

Se le proporciona el fichero /src/middlewares/ReviewMiddleware.js con los prototipos para sendas funciones userHasPlacedOrderInRestaurant, checkCustomerHasNotReviewed que ha de implementar.

```
const userHasPlacedOrderInRestaurant = async (req, res, next) => {
  try {
    const userId = req.user.id
    const restaurantId = req.params.restaurantId
    const order = await Order.findOne({where: { userId,restaurantId }})
    if (!order) {
      return res.status(409).json({
        message: 'You must have placed an order in this restaurant to write a review.'
      })
    }
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio

Necesito concentración

ali ali oooh esto con 1 coin me lo quito yo...

WUOLAH

```
    next()
  } catch (err) {
    res.status(500).json({ error: err.message })
  }
}

const checkCustomerHasNotReviewed = async (req, res, next) => {
  try {
    const userId = req.user.id
    const restaurantId = req.params.restaurantId
    const review = await Review.findOne({ where: {customerId: userId,restaurantId} })
    if (review) {
      return res.status(409).json({
        message: 'You have already reviewed this restaurant.'
      })
    }
    next()
  } catch (err) {
    res.status(500).json({ error: err.message })
  }
}
```

6. Cálculo de avgStars (2 puntos)

Implemente la lógica para calcular el atributo virtual avgStars en la función getAvgStars que encontrará en src/models/Restaurant.js. Sólo es necesario completar esta función para que el modelo incluya la propiedad virtual, el resto de cambios necesarios ya están incorporados.

```
async getAvgStars () {
   if (!this.reviews || this.reviews.length === 0) {
        return 0
   }
   const totalStars = this.reviews.reduce((sum, review) =>
           {return sum + review.stars}, 0)
   return totalStars / this.reviews.length
}
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.